

SOFTWARE ENGINEERING: A Practitioners Approach

01. software and software engineering
02. process models (the software process)
03. agile development (the software process)
04. principles that guide practice (modeling)
05. understanding requirements (modeling)
06. requirement modeling: scenarios, information, and analysis classes (modeling)
07. requirement modeling: flow, behavior, patterns, and webapps (modeling)
08. design concepts (modeling)
09. architectural design (modeling)
10. component-level design (modeling)
11. user interface design (modeling)
12. pattern-based design (modeling)
13. webapp design (modeling)
14. quality concepts (quality management)
15. review techniques (quality management)
16. software quality assurance (quality management)
17. software testing strategies (quality management)
18. testing conventional application (quality management)
19. testing object-oriented application (quality management)
20. testing web application (quality management)
21. formal modeling and verification (quality management)
22. software configuration management (quality management)
23. product metrics (quality management)
24. project management concepts (managing software projects)
25. process and project metrics (managing software projects)
26. estimation for software projects (managing software projects)
27. project scheduling (managing software projects)
28. risk management (managing software projects)
29. maintenance and re-engineering (managing software projects)
30. software process improvement (advance topics)
31. emerging trends in software engineering (advance topics)
32. concluding comments (advance topics)
33. an introduction to UML (appendix)
34. object-oriented concept (appendix)

FUNDAMENTAL OF SOFTWARE ENGINEERING

- | | |
|---|--------------------------|
| 1. Introduction | 13. Software Maintenance |
| 2. Software Life Cycle Models | 14. Software Reuse |
| 3. Software Project Management | 15. Emerging Trends |
| 4. Requirement Analysis and Specification | |
| 5. Software Design | |
| 6. Function-Oriented Software Design | |
| 7. Object Modelling Using UML | |
| 8. Object-Oriented Software Development | |
| 9. User Interface Design | |
| 10. Coding and Testing | |
| 11. Software Reliability and Quality Management | |
| 12. Computer Aid Software Engineering | |

Important Topics & Skills and Exam Focus

Chapter-1 (Page 1-18): Software and Software Engineering

Topics & Skills: Software definitions, application domains, legacy systems, software myths, process framework

Exam Focus: Definitions, myths, process framework

Chapter-2 (Page 20-35): Process Models

Topics & Skills: Generic process model, task sets, prescriptive models (Waterfall, Prototyping, Evolutionary, Unified)

Exam Focus: Model comparisons, task identification

Chapter-3 (Page 37-53): Agility and Process

Topics & Skills: Agile principles, Scrum, XP, Agile Manifesto, iterative development

Exam Focus: Agile principles, Scrum roles, Agile vs. traditional

Chapter-4 (Page 54-73): Recommended Process Model

Topics & Skills: Unified Process, phases, workflows, iterations, architecture-centric approach

Exam Focus: Phases, workflows, iterative development

Chapter-5 (Page 74-82): Human Aspects of Software Engineering

Topics & Skills: Team dynamics, communication, leadership, motivation, conflict resolution

Exam Focus: Team roles, communication strategies

Chapter-6 (Page 84-101): Principles That Guide Practice

Topics & Skills: Software engineering principles, best practices, ethical considerations

Exam Focus: Principles, best practices

Chapter-7 (Page 102-125): Understanding Requirements

Topics & Skills: Elicitation techniques, stakeholder analysis, requirements gathering methods

Exam Focus: Elicitation techniques, stakeholder analysis

Chapter-8 (Page 126-155): Requirements Modeling—A Recommended Approach

Topics & Skills: Use cases, scenarios, user stories, requirements specification techniques

Exam Focus: Use case diagrams, specification techniques

Chapter-9 (Page 156-180): Design Concepts

Topics & Skills: Abstraction, modularity, information hiding, cohesion, coupling, design patterns

Exam Focus: Design principles, patterns

Chapter-10 (Page 181-205): Architectural Design—A Recommended Approach

Topics & Skills: Architectural styles, patterns, views, components, connectors, architectural design process

Exam Focus: Architectural styles, design process

Chapter-11 (Page 206-232): Component-Level Design

Topics & Skills: Component design principles, modularity, interfaces, component testing

Exam Focus: Component design principles, testing

Chapter-12 (Page 233-263): User Experience Design

Topics & Skills: Usability, user interface design principles, interaction design, prototyping

Exam Focus: Usability principles, prototyping

Chapter-13 (Page 264-288): Design for Mobility

Topics & Skills: Mobile application design, constraints, performance, user experience, platform considerations

Exam Focus: Mobile design constraints, platform considerations

Chapter-14 (Page 289-308): Pattern-Based Design

Topics & Skills: Design patterns, creational, structural, behavioral patterns, pattern application

Exam Focus: Pattern types, application examples

Chapter-15 (Page 310-324): Quality Concepts

Topics & Skills: Software quality, quality assurance, quality control, quality metrics

Exam Focus: Quality definitions, metrics

Chapter-16 (Page 325-338): Reviews—A Recommended Approach

Topics & Skills: Formal reviews, walkthroughs, inspections, review process, roles

Exam Focus: Review types, process steps

Chapter-17 (Page 339-355): Software Quality Assurance

Topics & Skills: SQA activities, standards, audits, quality metrics, continuous improvement

Exam Focus: SQA activities, standards

Chapter-18 (Page 356-371): Software Security Engineering

Topics & Skills: Security principles, threat modeling, secure design, security testing

Exam Focus: Security principles, threat modeling

Chapter-19 (Page 372-394): Software Testing—Component Level

Topics & Skills: Unit testing, test cases, test drivers, stubs, boundary conditions, equivalence partitioning

Exam Focus: Unit testing techniques, test case design

Chapter-20 (Page 395-411): Software Testing—Integration Level

Topics & Skills: Integration testing strategies, top-down, bottom-up, sandwich, incremental, big bang

Exam Focus: Integration strategies, advantages/disadvantages

Chapter-21 (Page 412-436): Software Testing—Specialized Testing for Mobility

Topics & Skills: Mobile testing challenges, device fragmentation, performance testing, usability testing

Exam Focus: Mobile testing challenges, performance testing

Chapter-22 (Page 437-459): Software Configuration Management

Topics & Skills: Version control, change control, configuration identification, SCM tools

Exam Focus: SCM processes, tools

Chapter-23 (Page 460-488): Software Metrics and Analytics

Topics & Skills: Metrics types, measurement techniques, analytics, COCOMO, function points

Exam Focus: Metric types, COCOMO model

Chapter-24 (Page 490-503): Project Management Concepts

Topics & Skills: Project planning, scope, objectives, deliverables, project lifecycle

Exam Focus: Project planning steps, deliverables

Chapter-25 (Page 504-531): Creating a Viable Software Plan

Topics & Skills: Software project plan, work breakdown structure, scheduling, budgeting, resource allocation

Exam Focus: Project planning components, scheduling

Chapter-26 (Page 532-548): Risk Management

Topics & Skills: Risk identification, assessment, mitigation strategies, risk monitoring

Exam Focus: Risk management strategies

Chapter-27 (Page 549-566): A Strategy for Software Support

Topics & Skills: Software maintenance, support models, customer feedback, service level agreements

Exam Focus: Maintenance strategies, support models

Chapter-28 (Page 568-582): Software Process Improvement

Topics & Skills: SPI models, CMMI, Six Sigma, process assessment, continuous improvement

Exam Focus: SPI models, process assessment

Chapter-29 (Page 583-601): Emerging Trends in Software Engineering

Topics & Skills: AI in software engineering, cloud computing, microservices, DevOps, blockchain

Exam Focus: Emerging technologies, applications

Chapter-30 (Page 602-610): Concluding Comments

Topics & Skills: Future of software engineering, challenges, evolving practices

Exam Focus: Future trends, challenges

Study Tips for Exam Preparation

- **Focus on Chapter Summaries:** Each chapter concludes with a summary highlighting key concepts.
- **Practice Diagram Drawing:** For topics like UML, DFDs, and design patterns, practice drawing and interpreting diagrams.
- **Understand Real-World Applications:** Relate theoretical concepts to real-world scenarios to better grasp their practical implications.
- **Review End-of-Chapter Questions:** Engage with the review questions provided at the end of each chapter to test your understanding.

CHAPTER-1: SOFTWARE AND SOFTWARE ENGINEERING

Chapter-1 (Page 1-18): Software and Software Engineering

Topics & Skills: Software definitions, application domains, legacy systems, software myths, process framework

Exam Focus: Definitions, myths, process framework

Quick Look (Software and Software Engineering)

- What is it?
 - Computer software is the product that software professionals build and then support over long term
 - It encompasses programs that execute within a computer of any size and architecture, content that is presented as the computer program execute, and descriptive information in both hard copy and virtual forms that encompass virtually any electronic media
 - Software engineering encompasses a process, a collection of methods (practice) and an array of tools that allow professionals to build high-quality computer software
- Who does it?
 - Software engineers build and support software, and virtually everyone in the industrialized world uses it either directly or indirectly
- Why it is important?
 - Software is important because it affects nearly every aspect of our lives and has become pervasive in our commerce, our culture, and our everyday activities.
 - Software engineering is important because it enables us to build complex systems in a timely manner and with high quality
- What are the steps?
 - You build computer software like you build any successful product, by applying an agile, adaptable process that leads to a high-quality result that meets the needs of the people who will use the product.
 - You apply a software engineering approach
- What is the work product?
 - From the point of view of a software engineering, the work product is the set of programs, content (data), and other work products that are computer software
 - But from the user's viewpoint, the work product is the resultant information that somehow makes the user's world better
- How do I ensure that I've done it right?
 - Read the remainder of this book, select those ideas that are applicable to the software that you build, and apply them to your work

01. The Nature of the Software

- Today software takes on a dual role. It is a product, and at the same time, the vehicle for delivering product.
- As a product, it delivers the computer potential embodied by computer hardware or more broadly, by a network of computers that are accessible by local hardware
- As a vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs (software tools and environment)

- Software delivers the most important product of our time – information
- Whether it resides within a mobile phone or operates inside a mainframe computer, software is an information transformer – producing, managing, acquiring, modifying, displaying, or transmitting information that can be as simple as single bit or as complex as a multimedia presentation derived from data acquired from dozens of independent sources
- It transforms personal data (an individual financial transaction) so that the data can be more useful in a local context; It manages business information to enhance competitiveness; It provides a gateway to worldwide information networks (the internet), and provides the means for acquiring information in all of its forms.

- The role of computer software has undergone significant change
- Dramatic improvements in hardware performance, profound changes in computer architectures, vast increases in memory and storage capacity, and a wide variety of exotic input and output options, have all precipitated more sophisticated and complex computer-based systems

- Sophistication and complexity can produce dazzling results when a system succeeds, but they can also pose huge problems for those who must build complex systems.

- Today, a huge software industry has become a dominant factor in the economies of the industrialized world
- Teams of software specialists, each focusing on one part of the technology required to deliver a complex application, have replaced the lone programmer of an earlier era.
- And yet, the questions that were asked of the lone programmer are the same questions that are asked when modern computer-based systems are built
- These, and many other questions, are a manifestation of the concern about software and the manner in which it is developed – a concern that has led to the adoption of software engineering practice.

- These questions are –
 - Why does it take so long to get software finished?
 - Why are development costs so high?
 - Why can't we find all errors before we give the software to our customers?
 - Why do we spend so much time and effort maintaining existing programs?
 - Why do we continue to have difficulty in measuring processes as software is being developed and maintained?

02. [01] Defining Software

- A textbook description of software might take the following form, Software is –
 - Instructions (computer program) that when executed provided desired features, function, and performance
 - Data structures that enable the program to adequately manipulate information
 - Descriptive information in both hard copy and virtual forms that describe the operation and use of the software
- To improve your understanding, it's important to examine the characteristics of software that make it different from other things that human beings build.
- Software is a logical rather than a physical system element.
- Therefore, software has characteristics that are considerably different than those of hardware –
 - Software is developed or engineered; it is not manufactured in the classical sense
 - Software doesn't – wear out
 - Although the industry is moving toward component-based construction, must software continue to be custom built
- Software is developed or engineered; it is not manufactured in the classical sense
 - Although some similarities exist between software development and hardware manufacturing but two activities are fundamentally different
 - Software projects cannot be managed as if they were manufacturing projects.
 - In both activities, high quality is achieved through good design
 - But the manufacturing phase for hardware can introduce quality problems that are nonexistent (or easily corrected) for software.
 - Both activities are depended on people, but the relationship between – people applied and work accomplished is entirely different
 - Software costs are concentrated in engineering
- Software doesn't – Wear Out
 - Hardware exhibits relatively high failure rates early in its life (these failures are often attributable to design or manufacturing defects)
 - Defects are corrected and the failure rate drops to a steady-state level (hopefully, quite low) for some period of time
 - As time passes, the failure rate rises again as hardware components suffer from the cumulative effects of dust, vibration, abuse, temperature extremes, and many other environmental maladies
 - Stated simply, the hardware begins to wear out
 - Software is not susceptible to the environmental maladies that cause hardware to wear out
 - Undiscovered defects will cause high failure rates early in the life of a program
 - However, these are corrected and the failure rate drops to a steady-state level
 - However, the implication is clear – software doesn't wear out, but it does deteriorate

How Software does – Deteriorate

- During its life software will undergo changes
- As changes are made, it is likely that errors will be introduced, causing the failure rate to increase
- Before the failure rate returns to normal, another change is made, and the failure rate increases again
- Over time, the baseline or the lowest failure rate increases, indicating the software is deteriorating due to ongoing changes

Software doesn't have – Spare Part

- Another aspect of wear illustrates the difference between hardware and software
- When a hardware component wears out, it is replaced by a spare part
- There are no software spare parts

- Every software failure indicates an error in design or in the process through which design was translated into machine executable code
- Therefore, the software maintenance tasks that accommodate requests for change involve considerably more complexity than hardware maintenance

➤ Although the industry is moving toward component-based construction, most software continues to be custom build

- As an engineering discipline evolves, a collection of standard design components is created.
- Standard screws and off-the-shelf integrated circuits are only two of thousands of standard components that are used by mechanical and electrical engineers as they design new system.
- The reusable components have been created so that the engineer can concentrate on the truly innovative elements of a design, that is, the part of the design that represent something new.
- In the hardware world, component reuse is a natural part of the engineering process.
- In the software world, it is something that has only begun to be achieved on a broad scale.
- A software component should be designed and implemented so that it can be reused in many different programs
- Modern reusable components encapsulate both data and the processing that is applied to the data, enabling the software engineer to create new applications from reusable parts
- For example, today's interactive user interface is built with reusable components that enable the creation of graphics windows, pull-down menus, and a wide variety of interaction mechanisms
- The data structures and processing detail required to build the interface are contained within a library of reusable components for interface construction

03. [01] Software Application Domain

- Today's, seven broad categories of computer software present continuing challenges for software engineering –
 - System Software (Operating System, Device Drivers, System Utilities, File Management Tools)
 - Application Software (Microsoft Word, Google Chrome, Adobe Photoshop, Spotify)
 - Engineering / Scientific Software (MATLAB, AutoCAD, ANSYS, R)
 - Embedded Software (Microwave Oven, Washing Machine, Printer, ECU in Car, Smart TV, Router)
 - Product-line Software (Mobile OS, Banking Software, Point-of-sale systems)
 - Web Application (Gmail, Facebook, Google Docs, Amazon)
 - Artificial Intelligence Software (ChatGPT, Google Gemini – language mode, Siri, Alexa – Virtual Assistant, TensorFlow, PyTorch – AI Framework, Face Recognition Systems)
- Millions of software engineers worldwide are hard at work on software projects in one or more of these categories
- In some cases, new systems are being built, but in many others, existing applications are being corrected, adapted, and enhanced
- Past software engineers have made lasting contributions in all areas. Hopefully, this generation will do the same to help future engineers. Still, new challenges have appeared on the horizon –
 - Open-world computing (Autonomous vehicles or Self-Driving Car)
 - Net-sourcing (Cloud Computing Services like AWS, Company platform like Upwork, Toptal)
 - Open-source (Linux Operating, Mozilla Firefox, Apache HTTP Server)
- Each of these new challenges will undoubtedly obey the law of unintended consequences and have effects (for businesspeople, software engineers, and end users) that cannot be predicted today
- Software engineers can prepare by instantiating a process that is agile and adaptable enough to accommodate dramatic changes in technology and to business rules that are sure to come over the next decade
- System software
 - A collection of programs written to service other programs
 - Some system software (compiler, editor, and file management utilities) processes complex, but determinate, information structures
 - Other system applications (operating system components, drivers, networking software, telecommunication processors) process largely indeterminate data
 - In early case, the systems software area is characterized by heavy interaction with computer hardware; heavy usage by multiple users; concurrent operation that requires scheduling, resource sharing, and sophisticated process management; complex data structures; and multiple external interfaces.
- Application software
 - Stand-alone programs that solve a specific business need.
 - Applications in this area process business or technical data in a way that facilitates business operations or management/technical decision making.
 - In addition to conventional data processing applications, application software is used to control business functions in real time (point-of-sale transaction processing, real-time manufacturing process control)
- Engineering / Scientific software
 - Has been characterized by number crunching algorithms
 - Applications range from astronomy to volcanology, from automotive stress analysis to space shuttle orbital dynamic, and from molecular biology to automated manufacturing
 - However, modern applications within the engineering / scientific area are moving away from conventional numerical algorithms
 - Computer-aided design, system simulation, and other interactive applications have begun to take on real-time and even system software characteristics

- Embedded software
 - Resides within a product or system and it's used to implement and control features and functions for the end user and for the system itself
 - Embedded software can perform limited and esoteric functions (key pad control for a microwave oven) or provide significant function and control capability (digital functions in an automobile such as fuel control, dashboard display, and braking systems)
- Product-line software
 - Designed to provide a specific capability for use by many different customers
 - Product-line software can focus on a limited and esoteric marketplace (inventory control system) or address mass consumer markets (word processing, spreadsheet, computer graphics, multimedia, entertainment, database management, and personal and business financial application)
- Web Application
 - Call WebApps, this network-centric software category spans a wide array of applications
 - In their simplest form, WebApps can be little more than a set of linked hypertext files that present information using text and limited graphics
 - However, as Web 2.0 emerges, WebApps are evolving into sophisticated computing environments that not only provide stand-alone features, computing functions, and content to the end user, but also are integrated with corporate databases and business applications
- Artificial Intelligence software
 - Makes use of nonnumerical algorithm to solve complex problems that are not amenable to computation or straightforward analysis.
 - Applications within this area include robotics, expert systems, pattern recognition (image and voice), artificial neural networks, theorem proving, and game playing
- Open-world Computing
 - The rapid growth of wireless networking may soon lead to true pervasive, distributed computing.
 - The challenge for software engineers will be to develop systems and application software that will allow mobile device, personal computers, and enterprise systems to communicate across vast networks
- Net-sourcing
 - The World Wide Web is rapidly becoming a computing engine as well as a content provider
 - The challenge for software engineers is to architect simple (personal financial planning) and sophisticated applications that provide a benefit to targeted end-user markets worldwide
- Opensource
 - A Growing trend that results in distribution of source code for systems applications (operating system, database, and development environments) so that many people can contribute to its development
 - The challenge for software engineers is to build source code that is self-descriptive, but more importantly, to develop techniques that will enable both customers and developers to know what changes have make and how those changes manifest themselves within the software

04. [01] Legacy Software

- Hundreds of thousands of computer programs fall into one of the seven broad application domains discussed in the preceding subsection
- Some of these are state-of-the-art software-just released to individuals, industry, and government
- But other programs are older, in some cases much older and these often referred to as legacy software
- The goal of modern software engineering is to devise methodology that are founded on the notion of evolution; that is, the notion that software system continually change, new software systems are build from the old ones, and ... all must interoperate and cooperate with each other

- Unfortunately, there is somethings one additional characteristic that is present in legacy software – poor quality, sometimes have inextensible designs, convoluted code, poor and nonexistent documentation, test cases and results that were never archived, a poorly managed change history – the list can be quite long
- And yet, these systems support core business functions and are indispensable to the business
- If legacy software meets the needs of its users and run reliably, it isn't broken and does not need to be fixed
- However, as time passes, legacy systems often evolve or must be reengineered for one or more of the following reasons so that it remains viable into the future –
 - The software must be adapted to meet the needs of new computing environments or technology
 - The software must be extended to make it interoperable with other more modern systems or databases
 - The software must be enhanced to implement new business requirements
 - The software must be re-architected to make it viable within a network environment

- Legacy software (A Description)
 - Legacy Software systems ... were developed decades ago and have been continually modified to meet changes in business requirements and computing platforms.
 - The proliferation of such systems is causing headaches for large organizations who find them costly to maintain and risky to evolve
 - Many legacy systems remain supportive to core business functions and are indispensable to the business
 - Hence, legacy software is characterized by longevity and business critically

05. The Unique Nature of WebApps

- In the early days of the World Wide Web, websites consisted of little more than a set of linked hypertext files that presented information using text and limited graphics
- As time passed, the augmentation of HTML by development tools (XML, Java) enables web engineers to provide computing capability along with information content and Web-based systems and applications (WebApps) where born
- Today, WebApps have evolved into sophisticated computing tools that not only provide stand-alone function to the end user, but also have been integrated with corporate database and business applications

- WebApps are one of a number of distinct software categories, and yet, it can be argued that WebApps are different
- Web-based systems and applications involve a mixture between – print publishing and software development,
 - Between marketing and computing,
 - Between internal communication and external relations,
 - And between art and technology

- The following attributes are encountered in the vast majority of WebApps –
 - Network intensiveness
 - Concurrency (Correspondence)
 - Unpredictable load
 - Performance
 - Availability
 - Data driven
 - Content sensitive
 - Continuous evolution
 - Immediacy
 - Security

- Network intensiveness
 - A WebApp resides on a network and must serve the needs of a diverse community of clients
 - The network may enable world-wide access and communication (internet) or more limited access and communication (a corporate intranet)
- Concurrency
 - A large number of users may access the WebApp at one time
 - In many cases, the patterns of usage among end users will vary greatly
- Unpredictable Load
 - The number of users of the WebApp may vary by orders of magnitude from day to day
- Performance
 - If a WebApp user must wait too long (for access, for server-side processing, for client-side formatting and display), he or she may decide to go elsewhere
- Availability
 - Although expectation of 100% availability is unreasonable, users of popular WebApps often demand access on a 24/7/365 basis
 - Users in Australia or Asia might demand access during times when traditional domestic software application in North America might be taken off-line for maintenance
- Data driven
 - The primary function of many WebApps is to user hypermedia to present (text, graphics, audio and video) content to the end user
 - In addition, WebApps are commonly used to access information that exists on data-base that are not integral part of the Web-based environment (e-commerce and financial application)
- Content sensitive

- The quality and aesthetic nature of content remains an important determinant of the quality of a WebApps
- Continuous evolution
 - Unlike conventional application software that evolves over a series of planned, chronologically spaced releases, web application evolves continuously
 - It is not unusual for some WebApps (specially their content) to be updated on a minute-by-minute schedule or for content to be independently computed for each request
- Immediacy
 - Although immediacy – the compelling need to get software to market quickly – is a characteristic of many application domain, WebApps often exhibit a time-to-market that can be a matter of a few days or weeks
- Security
 - Because WebApps are available via network access, it is difficult, if not impossible, to limit the population of end users who may access the application
 - In order to protect sensitive content and provide secure modes of data transmission, strong security measures must be implemented throughout the infrastructure that supports a WebApps and within the application itself
- Aesthetics
 - An undeniable part of the appeal of a WebApp is its look and feel
 - When an application has been designed to market or sell products or ideas, aesthetics may have as much to do with success as technical design

06. Software Engineering

- In order to build software that is ready to meet the challenges of the twenty-first century, you must recognize a few simple realities –
 - We must work together to clearly understand the problem before creating the software
 - Designing the system becomes one of the most important steps
 - Software must be built to be very reliable and of high quality
 - Software should be easy to update, fix, and improve
- These simple realities lead to one conclusion – software in all of its forms and across all of its application domains should be engineered and that leads us to the subject – software engineering

- We must work together to clearly understand the problem before creating the software
 - Software has become deeply embedded in virtually every aspect of our lives, and as a consequence, the Number of people who have an interest in the features and functions provided by a specific application has grown dramatically
 - When a new application or embedded system is to build, many voices must be heard
 - And it sometimes seems that each of them has a slightly different idea of what software features and functions should be delivered
 - It follows that a connected effort should be made to understand the problem before a software solution is development

- Designing the system becomes one of the most important steps
 - The information technology requirements demanded by individuals, business, and governments grow increasing complex with each passing year
 - Large teams of people now create computer programs that were once built by a single individual
 - Sophisticated software that was once implemented in a predictable, self-contained, computing environment is now embedded inside everything from customer electronics, to medical devices to weapons systems
 - The complexity of these new computer-based systems and products demands careful attention to the interaction of all system elements
 - It follows that design becomes a pivotal activity

- Software must be built to be very reliable and of high quality
 - Individual, businesses, and governments increasingly rely on software for strategic and tactical decision making as well as day-to-day operations and control
 - If the software fails, people and major enterprises can experience anything from minor inconvenience to catastrophic failures
 - It follows that software should exhibit high quality

- Software should be easy to update, fix, and improve
 - As the perceived value of a specific application grows, the likelihood is that its user base and longevity will also grow
 - As its user base and time-in-use increase, demands for adaption and enhancement will also grow
 - It follows that software should be maintainable

➤ Definition of Software Engineering

■ Definition

- Software Engineering is the establishment and use of sound engineering principles in order to obtain economically software that is reliable and work efficiently on real machines

■ Definition of IEEE

- Software Engineering – (1) The application of a systematic, disciplined, qualifiable approach to the development, operation, and maintenance of software, that is, the application of engineering of software. (2) The study of approaches as in (1)

➤ Lack of these definitions

■ Definition

- It says little about the technical aspects of software quality; it does not directly address the need for customer satisfaction or timely product delivery
- It omits mention of the importance of measurement and metrics
- It does not state the importance of an effective process

■ yet, this definition provides a baseline, but questions that continue to challenge software engineering

- What are the sound engineering principles that can be applied to computer software development?
- How do we economically build software so that it is reliable?
- What is required to create computer programs that work efficiently on not one but many different real machines?

■ Definition of IEEE

- And yet, a systematic, disciplined, and quantifiable approach applied by one software team may be burdensome to another
- We need discipline, but we also need adaptability and agility

➤ Software engineering is a layered technology

- Any engineering approach (software engineering) must rest on an organizational commitment to quality
- Total quality management (six sigma) should be a continuous process improvement culture, and it is this culture that ultimately leads to the development of increasingly more efficient approaches to software engineering
- The bedrock that supports software engineering is a quality focus
- The foundation for software engineering is the process layer
- The software engineering process is the glue that hold the technology layers together and enables rational and timely developed of computer software
- Process defines a framework that must be established for effective delivery of software engineering technology
- The software process forms the basis for management control of software projects and establishes the context in which technical methods are applied, work products (models, documents, data, reports, forms) are produced, milestones are established, quality is ensured, and change is properly managed

➤ Software Engineering methods and Tools

- Software engineering methods provide the technical how-to's for building software
- Methods encompass a broad array of tasks that include communication, requirements analysis, design modeling, program construction, testing, and support.
- Software engineering methods rely on a set of basic principles that govern each area of the technology and include modeling activities and other descriptive technology
- Software engineering tools provide automated or semiautomated support for the process and methods
- When tools are integrated so that information created by one tool can be used by another, a system for support of software development, called computer-aid software engineering, is established

07. The Software Processes

➤ Process

- A process is a collection of activities, actions, and tasks that are performed when some work product is to be created
- An activity strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied
- An action (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., architectural design model)
- A task focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome

➤ Context of Software Engineering

- In the context of software engineering, a process is not a rigid prescription for how to build computer software.
- Rather, it is an adaptable approach that enables the people doing the work (the software team) to pick and choose the appropriate set of work actions and tasks
- The intent is always to deliver software in a timely manner and with sufficient quality to satisfy those who have sponsored its creation and those who will use it

➤ Process framework activities

- A process framework establishes the foundation for a complete software engineering process by identifying a small number of framework activities that are applicable to all software project, regardless of their size or complexity
- In addition, the process framework encompasses a set of umbrella activities that are applicable across the entire software process
- A generic process framework for software engineering encompasses five activities
 - Communication
 - Planning
 - Modeling
 - Construction
 - Deployment
- These five generic framework activities can be used during the development of small, simple program, the creation of large web applications, and for the engineering of large, complex computer-based systems
- The details of the software process will be quite different in each case, but the framework activities remain the same

➤ Process iteration

- For many software projects, framework activities are applied iteratively as a project progresses
- That is, communication, planning, modeling, construction, and deployment are applied repeatedly through a number of project iterations
- Each project iteration produces a software increment that provides stakeholders with a subset of overall software features and functionality
- As each increment is produced, the software becomes more and more complete

➤ Umbrella activities of Process

- Software engineering process framework activities are complemented by a number of umbrella activities
- In general, umbrella activities are applied throughout a software project and help a software team manage and control progress, quality, change, and risk
- Typical umbrella activities include

- Software project tracking and control
- Risk management
- Software quality assurance
- Technical reviews
- Measurement
- Software configuration management
- Reusability management
- Work product preparation and production

➤ Process agility and adaptability

- the software engineering process is not a rigid prescription that must be followed dogmatically by a software team
- rather, it should be agile and adaptable (to the problem, to the project, to the team, and to the organizational culture)
- Therefore, a process adopted for one project might be significantly different than a process adopted for another project.
- Among the differences are –
 - Overall flow of activities, actions, and tasks and the interdependencies among them
 - Degree to which actions and tasks are defined within each framework activity
 - Degree to which work products are identified and required
 - Manner in which quality assurance activities are applied
 - Manner in which project tracking and control activities are applied
 - Overall degree of detail and rigor with which the process is described
 - Degree to which the customer and other stakeholders are involved with the project
 - Level of autonomy given to the software team
 - Degree to which team organization and roles are prescribed

08. Software Engineering Practice

- Generic concept and principles
 - A generic software process model composed of a set of activities that establish a framework for software engineering practice
 - Generic framework activities – communication, planning, modeling, construction, and deployment – and umbrella activities establish a skeleton architecture for software engineering work
 - A basic understanding of the generic concepts and principles that apply to framework activities
 - The essence of practice
 - General principles
- The essence of practice
 - Before modern computers existed, the essence of problem solving, and consequently, the essence of software engineering practice –
 - Understanding the problem (communication and analysis)
 - Plan a solution (modeling and software design)
 - Carry out the plan (code generation)
 - Examine the result for accuracy (testing and quality assurance)
 - In this context of software engineering, these commonsense steps lead to a series of essential questions
 - It shouldn't surprise you that much of this approach is common sense
 - In fact, it's reasonable to state that a commonsense approach to software engineering will never lead you astray
- General principles
 - The dictionary defines the word principle as – an important underlying law or assumption required in a system of thought
 - Throughout this book I'll discuss principles at many different levels of abstraction.
 - Some focus on software engineering as a whole, others consider a specific generic framework activity (e.g., communication), and still others focus on software engineering action (e.g., architectural design) or technical task (e.g., write a usage scenario)
 - Regardless of their level of focus, principles help you establish a mind-set for solid software engineering practice.
 - They are important for that reason
 - The proposed seven principles that focus on software engineer practice as a whole
 - They are reproduced in the following –
 - The first principles: The reason it all exists
 - The second principles: KISS (Keep It Simple, Stupid!)
 - The third principles: Maintain the version
 - The fourth principles: What you produce, Others will consume
 - The fifth principles: Be open to the future
 - The sixth principles: Plan ahead for reuse
 - The seventh principles: Think
 - If every software engineer and every software team simply followed these seven principles, many of the difficulties experienced in building complex computer-based systems would be eliminated

09. Software Myths

- Software myths – erroneous beliefs about software and the process that is used to build it – can be traced to the earliest days of computing
- Myths have a number of attributes that make them insidious
- For instance, they appear to be reasonable statements of fact (sometimes containing elements of truth), they have an intuitive feel, and they often promulgated by experience practitioners who – know the score
- Today, most knowledge software engineering professionals recognize myths for what they are – misleading attribute that have caused serious problems for managers and practitioners alike
- However, old attribute and habits are difficult to modify, and remnants of software myths remain
- Management myths
 - Managers with software responsibility, like managers in most disciplines, are often under pressure to maintain budgets, keep schedules from slipping, and improve quality
 - Like a drawing person who grasps at a straw, a software manager often grasps at belief in a software myth, if that belief will lessen the pressure (even temporarily)
 - Myth 1:
 - We already have a book that's full of standards and procedures for building software
 - Won't that provide my people with everything they need to know?
 - Myth 2:
 - If we get behind schedule, we can add more programmers and catch up (sometimes called the – Mongolian horde – concept
 - Myth 3:
 - If I decide to outsource the software project to a third party, I can just relax and let that firm build it
 - Reality 1:
 - The book of standards may very well exist, but is it used?
 - Are software practitioners aware of its existence?
 - Does it reflect modern software engineering practice?
 - Is it complete?
 - Is it streamlined to improve time-to-delivery while still maintaining a focus on quality?
 - In many cases, the answer to all of these questions is – no
 - Reality 2:
 - Software development is not a mechanistic process like manufacturing
 - Adding people to a late software project makes it later
 - At first, this statement may seem counterintuitive
 - However, as new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort
 - People can be added but only in a planned and well-coordinated manner
 - Reality 3:
 - If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it out-source software projects.
- Customer myths
 - A customer who requests computer software may be a person at the next desk, a technical group down the hall, the marketing/sales department, or an outside company that has requested software under contract.
 - In many cases, the customer believes myths about software because software managers and practitioners do little to correct misinformation
 - Myths lead to false expectations (by the customer) and, ultimately, dissatisfaction with the developer
 - Myth 1:
 - A general statement of objectives is sufficient to begin writing programs – we can fill in the details later
 - Myth 2:

- Software requirements continually change, but change can be easily accommodated because software is flexible
- Reality 1:
 - Although a comprehensive and stable statement of requirements is not always possible, an ambiguous – statement of objectives – is a recipe for disaster
 - Unambiguous requirements (usually derived iteratively) are developed only through effective and continuous communication between customer and developer
- Reality 2:
 - It is true that software requirements change, but the impact of change varies with the time at which it is introduced
 - When requirements changes are requested early (before design or code has been started), the cost impact is relatively small
 - However, as time passes, the cost impact grows rapidly – resources have been committed, a design framework has been established, and change can cause upheaval that requires additional resources and major design modification

➤ Practitioner's myths

- Myths that are still believed by software practitioners have been fostered by over 50 years of programming culture
- During the early days, programming was viewed as an art form
- Old ways and attitudes die hard
- Myth 1:
 - Once we write the program and get it to work, our job is done
- Myth 2:
 - Until I get the program – running – I have no way of assessing its quality
- Myth 3:
 - The only deliverable work product for a successful project is the working program
- Myth 4:
 - Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down
- Reality 1:
 - Someone once said that – the sooner you begin – writing code – the longer it'll take you to get done
 - Industry data indicate that between 60 and 80 percent of all effort expended on software will be expended after it is delivered to the customer for the first time
- Reality 2:
 - One of the most effective software quality assurance mechanisms can be applied from the inception of a project – the technical review
 - Software reviews are a – quality filter – that have been found to be more effective than testing for finding certain classes of software defects
- Reality 3:
 - A working program is only one part of software configuration that includes many elements
 - A variety of work products (model, documents, planes) provide a foundation for successful engineering and, more important, guidance for software support
- Reality 4:
 - Software engineering is not about creating documents
 - It is about creating a quality product
 - Better quality leads to reduced rework
 - And reduced rework results in faster delivery times

➤ Many software professionals recognize the fallacy of the myths just described

- Regrettably, habitual attitudes and methods foster poor management and technical practices, even when reality dictates a better approach

- Recognition of software realities is the first step toward formulation of practical solutions for software engineering

Thanks! Let's break down this important text into clear parts so you can understand the **concept**, **master the topic**, and even prepare for **university-level interviews**.

1. Simplified Explanation & Summary

Main Concept:

Modern software development is complex and critical because software is deeply integrated into every part of society. As a result, understanding needs, designing carefully, and ensuring quality are essential.

Key Points:

► Software is Everywhere:

- Software is now embedded in daily life – smartphones, appliances, cars, health devices, etc.
- Many people care about what the software does (stakeholders), and each may have different expectations.
- So, before building software, developers must deeply understand the problem (i.e., strong requirements gathering).

► Increasing Complexity:

- Software problems are no longer simple – the tech needs of businesses, governments, and people are growing.
- What used to be built by one person now needs a large team.
- Software is integrated into complex systems like medical devices or defense tools.
- This requires **thoughtful design**, as system components interact in intricate ways.

► Dependence on Software:

- Everyone depends on software for decisions and operations.
- Failures can cause anything from small issues to huge disasters.
- Hence, the software must be of **high quality** (reliable, secure, well-tested, and efficient).



2. Real-world Examples

• Example 1: Medical Devices

A pacemaker relies on embedded software. If the software malfunctions, it could risk a patient's life. So, quality and design are vital.

• Example 2: Banking Systems

Software is used to process millions of transactions. If the system fails, it might cause financial chaos.

• Example 3: Google Maps

It gathers input from multiple stakeholders – users, traffic authorities, advertisers – and needs to balance all their needs.



3. Top University & Interview Questions (for Professionals)



University-Level (MIT, Stanford, etc.):

1. How do you manage conflicting requirements from multiple stakeholders in a software project?
2. What design methodologies help manage complexity in large-scale software systems?
3. Discuss the importance of risk analysis in high-reliability software systems (e.g., healthcare, defense).



Industry/Big Tech Interviews (Google, Microsoft, Amazon):

1. Describe how you'd design software for a device that cannot afford to fail (like an airplane autopilot system).
2. How do you ensure software quality when working on a large team with frequent updates?
3. Explain a time you dealt with ambiguous or conflicting feature requirements – how did you resolve it?



4. Phrases, Idioms & Synonyms (with Application)

Phrase / Idiom	Meaning	Synonyms	Example Usage
Embedded in virtually every aspect	Deeply integrated into all areas	Ingrained, interwoven	"AI is embedded in virtually every aspect of digital life today."
Voices must be heard	Everyone's opinions should be considered	Input considered, perspectives valued	"In UX design, all user voices must be heard."

Phrase / Idiom	Meaning	Synonyms	Example Usage
Concerted effort	Coordinated and focused attempt	Joint effort, collaborative action	"A concerted effort is needed to improve software security."
Pivotal activity	Extremely important part	Crucial task, central action	"Design is a pivotal activity in system architecture."
Day-to-day operations	Routine daily activities	Routine functions, regular tasks	"ERP systems support business day-to-day operations."
Catastrophic failure	A complete, disastrous breakdown	Total failure, major breakdown	"NASA avoids catastrophic failure through intense system testing."

✅ Summary Takeaway (Cheat Sheet)

Core Need	Action Required
Stakeholders vary	Strong requirement analysis
Systems are complex	Careful system design
Software is critical	High software quality assurance

Would you like to go deeper into one of the sub-topics like **design methodology**, **requirements engineering**, or **software quality standards (like ISO or CMMI)**?
 Let me know – I can build a full learning path with study materials too!

Yes, **generic software** is one type of software. There are several other major types of software based on how they are **developed** and **used**. Here's a classification with explanation:

✅ 1. Generic Software

- **Definition:** Software developed to be sold to a broad market.
- **Users:** General public or organizations with similar needs.

- **Examples:** Microsoft Office, Adobe Photoshop, VLC, Google Chrome.
 - **Cost:** One-time purchase or subscription.
-

✓ **2. Custom (or Bespoke) Software**

- **Definition:** Software developed for a **specific client or purpose**.
 - **Users:** One client or organization.
 - **Examples:** Banking systems, hospital management systems developed for a specific institution.
 - **Cost:** Higher, due to customization and exclusive development.
-

✓ **3. Freeware**

- **Definition:** Software that is free to use, but the source code is **not open**.
 - **Examples:** Skype, Adobe Acrobat Reader.
-

✓ **4. Open-source Software**

- **Definition:** Software whose source code is publicly available and can be modified.
 - **Examples:** Linux, GIMP, Firefox.
-

✓ **5. Shareware**

- **Definition:** Software you can try for free for a limited time or with limited features.
 - **Examples:** WinRAR, some games or editing tools with trial versions.
-

✓ **6. Embedded Software**

- **Definition:** Software embedded in hardware devices to control them.
 - **Examples:** Software in microwave ovens, washing machines, cars, medical devices.
-

✓ **7. System Software**

- **Definition:** Software that manages hardware and system resources.
 - **Examples:** Operating systems (Windows, Linux, macOS), device drivers.
-

✓ **8. Application Software**

- **Definition:** Software designed to help users perform specific tasks.
 - **Examples:** MS Word (word processing), Excel (spreadsheet), Zoom (video calling).
-

✓ **9. Utility Software**

- **Definition:** Software that helps maintain, analyze, or optimize a computer.
 - **Examples:** Antivirus software, disk cleanup tools.
-

✓ **10. Web-based Software (SaaS)**

- **Definition:** Software accessed via a web browser; runs on remote servers.
 - **Examples:** Google Docs, Canva, Dropbox.
-

Let me know if you'd like this as a **chart**, or want it translated into **Bengali**.